Chat with Retail Documents (RAG)

Course Name: Gen-AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Suzal Hasam Ali Devani	EN22IT301111
2	Aditi SIngh	EN24CA5030008
3	Naina Sahu	EN24CA5030106
4	Aryan Gangrade	EN22IT301023
5	Ayushi Jat	EN22IT301028
6	Preeti Prajapat	EN24CA5030131

Group Name : Group 11 D12

Project Number: GAI 21

Industry Mentor Name: Mr. Yash Bansal

University Mentor Name: Prof. Ajaj Khan

Academic Year:2025-26

Problem Statement & Objectives

1. Problem Statement
2. Project Objectives
3. Scope of the Project

Proposed Solution

1. Key features *(Just mention key features here no need to go into details)*
2. Overall Architecture / Workflow
3. Tools & Technologies Used (*If applicable)*

Results

Add the below details here:

1. Screenshots / outputs
2. Reports / dashboards / models
3. Key outcomes

Conclusion

Mention a brief conclusion about your project summing up everything you have worked on and the key learning.

Future Scope & Enhancements

1. Problem Statement

In the retail industry, organizations generate and manage a vast amount of unstructured data in the form of documents such as product manuals, inventory reports, policy documents, sales reports, and operational guidelines. These documents often contain critical business information, but retrieving specific insights from them is a challenging and time-consuming task.

Traditional keyword-based search systems fail to capture the semantic meaning of queries, resulting in irrelevant or incomplete results. Additionally, manual analysis of documents is inefficient, especially when dealing with large-scale data.

With the advancement of Artificial Intelligence and Natural Language Processing, there is a growing need for intelligent systems that can understand the context of user queries and provide precise answers.

Another major problem is that traditional search systems only work based on keywords and do not understand the meaning of the question. If the exact keyword is not present, the system may not return the correct result. Therefore, there is a need for an intelligent system that can understand user questions in natural language and provide accurate answers directly from the documents.

This project solves this problem by using Retrieval-Augmented Generation (RAG), which combines document retrieval and language generation. The system retrieves relevant information from the uploaded documents and then generates a proper answer using a language model. This makes document searching faster, easier, and more efficient.

The problem can be further decomposed into the following specific challenges:

- Semantic Mismatch: Keyword search does not understand natural language, causing users to miss relevant information even when it clearly exists in documents.
- Information Silos: Different departments maintain separate document repositories, making cross-document queries nearly impossible without manual effort.
- Response Reliability: General-purpose AI assistants that rely solely on pre-trained knowledge are prone to hallucination — generating plausible-sounding but factually incorrect responses.
- Scalability: As a retail business grows, the volume of documentation grows proportionally, making manual retrieval increasingly unsustainable.
- Context Sensitivity: Business queries are often contextual (e.g., "What is the policy for orders placed during a sale?") and require the system to consider multiple parts of a document simultaneously.

2. Project Objectives

The main objective of this project is to develop an intelligent document question-answering system for the retail domain using Retrieval-Augmented Generation. The system allows users to upload retail documents and ask questions in natural language, and the system provides accurate answers based on the uploaded documents.

Another objective of this project is to implement a complete RAG pipeline that includes document loading, text extraction, text chunking, vector embedding, vector storage, similarity search, and answer generation using a Large Language Model. The project also aims to reduce manual effort in searching documents and improve the speed of information retrieval.

Objective	Description
Document Ingestion	Enable users to upload retail documents in PDF format for automated processing.
Text Extraction	Accurately extract textual content from uploaded documents using robust parsing tools.
Semantic Chunking	Divide documents into meaningful, overlapping text segments to preserve context during retrieval.
Vector Embedding	Convert text chunks into dense numerical vector representations that capture semantic meaning.
Vector Storage	Persist embeddings in a high-performance vector database optimised for similarity search.
Semantic Retrieval	Retrieve the most contextually relevant document chunks for any given user query.
LLM Integration	Pass retrieved context to a Large Language Model to generate accurate, human-readable answers.
Hallucination Reduction	Ground all responses strictly in retrieved document content to prevent fabricated outputs.
User Interface	Provide a clean, interactive web interface for document upload and conversational querying.

2. Scope of the Project

The scope of this project is limited to document-based question answering in the retail domain. The system works only on the documents uploaded by the user and does not use external information from the internet.

In Scope

The following capabilities fall within the scope of this project:

- Processing of retail domain documents including product manuals, policy documents, inventory reports, and operational guidelines.
- Support for PDF as the primary document format, with reliable text extraction even from multi-page, multi-column PDFs.
- Implementation of a complete RAG pipeline: document upload → text extraction → chunking → embedding → vector storage → similarity retrieval → LLM response generation.
- Deployment of a responsive web application with a frontend (HTML/CSS/JS) and a RESTful backend API (FastAPI).
- Integration with ChromaDB for vector storage and SentenceTransformers for embedding generation.
- Use of Groq API for fast, low-latency LLM inference with models such as Llama 3.

Out of Scope

- Real-time integration with external data feeds, live inventory systems, or enterprise ERPs.
- Training or fine-tuning of custom language models from scratch.
- Handling multimedia file formats such as images, audio recordings, or video content.
- Multi-user authentication, role-based access control, or enterprise-grade security features (planned for future scope).
- Editing, annotating, or modifying uploaded documents within the application.

Known Limitations

- Answer accuracy is directly dependent on the quality and completeness of the uploaded documents.
- Documents with complex layouts, tables, or scanned images may not be parsed with complete fidelity.
- The system is stateless between sessions — previously uploaded documents are not persisted across application restarts without additional configuration.
- Performance is sensitive to the embedding model chosen; lighter models may sacrifice accuracy for speed.

3. Proposed Solution

Retrieval-Augmented Generation (RAG) is a hybrid AI architecture that combines the strengths of two complementary paradigms: dense information retrieval and neural text generation. It was introduced to address a fundamental limitation of standalone large language models (LLMs): while LLMs are extraordinarily capable at generating fluent and contextually coherent text, their knowledge is fixed at the point of training and cannot be dynamically updated or grounded in specific external documents.

In a RAG system, when a user poses a question, the system does not simply ask the LLM to answer from memory. Instead, it first searches an external knowledge base — in this case, the collection of retail documents uploaded by the user — to identify the most relevant passages. These passages are then passed to the LLM as context, alongside the user's original question. The LLM generates its response based on this provided context, ensuring that the answer is grounded in real, document-specific information rather than generalised or potentially outdated pre-training knowledge.

This architecture delivers three critical advantages for the retail use case: (1) answers are factually grounded in the organisation's own documents; (2) the system can be updated simply by uploading new documents, without any model retraining; and (3) the risk of hallucination is significantly reduced because the model is constrained to synthesise information from retrieved passages.

The proposed solution is based on the **Retrieval-Augmented Generation (RAG)** architecture, which combines information retrieval with generative AI.

Concept Overview:

RAG enhances traditional LLM capabilities by grounding responses in external knowledge sources (documents). Instead of relying solely on pre-trained knowledge, the system retrieves relevant information dynamically and uses it to generate responses.

Solution Workflow:

1. **Document Upload:**
   Users upload retail documents in PDF format.
2. **Text Extraction:**
   Text is extracted using PDF parsing libraries.
3. **Text Preprocessing:**
   - Removal of noise

4. **Chunking:**
   Large text is divided into smaller chunks to improve retrieval efficiency.
5. **Embedding Generation:**
   Each chunk is converted into a vector representation using embedding models.
6. **Vector Storage:**
   Embeddings are stored in a vector database.
7. **Query Processing:**
   User queries are converted into embeddings.
8. **Similarity Search:**
   Relevant document chunks are retrieved using similarity metrics (e.g., cosine similarity).
9. **Context Injection:**
   Retrieved chunks are passed as context to the LLM.
10. **Response Generation:**
    The LLM generates a final answer based on retrieved context.

4. Key Features

Feature	Description
Document Upload System	A clean drag-and-drop interface allowing users to upload one or more retail PDF documents. Files are validated, stored temporarily, and queued for the processing pipeline.
Automated Text Extraction	Uses PyPDF to extract raw text from each page of an uploaded PDF. Handles multi-page documents and preserves paragraph structure where possible.
Intelligent Text Chunking	Splits extracted text into overlapping chunks of approximately 500 characters. Overlap ensures that information spanning chunk boundaries is not lost during retrieval.
Semantic Embedding Generation	Transforms each text chunk into a high-dimensional vector using the SentenceTransformers library (all-MiniLM-L6-v2 model), capturing semantic meaning beyond keywords.
ChromaDB Vector Storage	Persists all chunk embeddings in ChromaDB, a lightweight in-process vector database optimised for cosine similarity search at low latency.
Similarity-Based Retrieval	When a user submits a query, it is converted to an embedding and compared against all stored chunk vectors. The top-k most similar chunks are retrieved.
LLM-Powered Answer Generation	Retrieved chunks and the user question are passed to the Groq API (Llama 3 model). The LLM synthesises a coherent, contextually accurate response.
Hallucination Guardrails	The system prompt instructs the LLM to answer strictly from the provided context, refusing to speculate beyond the document content.
Interactive Chat Interface	A conversational UI with suggested query buttons, chat history display, and real-time response streaming for an intuitive user experience.
Full-Stack Architecture	Decoupled frontend (HTML/CSS/JS) and backend (FastAPI) communicating via REST API, enabling independent scaling and deployment.

6. Overall Architecture / Workflow

The system architecture is based on the Retrieval-Augmented Generation model. First, the user uploads a document such as a PDF file. Then the system extracts text from the document and divides the text into smaller chunks so that the information can be processed properly. After that, these text chunks are converted into vector embeddings using an embedding model. These embeddings are stored in a vector database such as FAISS.

When a user asks a question, the system converts the question into an embedding and searches the vector database to find the most similar text chunks. These relevant text chunks are then passed to a Large Language Model, which reads the retrieved content and generates a final answer in natural language. The answer is then displayed to the user in the chat interface. This entire process is called the RAG pipeline.

Layer 1: Input Layer

The input layer handles all user interactions. There are two primary input types:

- Document Upload: The user selects one or more PDF files from their local device via the web frontend. The file is transmitted to the FastAPI backend as a multipart form-data request.
- Natural Language Query: The user types a question into the chat input field. The question is sent to the backend as a JSON payload via a POST request to the /ask endpoint.

Layer 2: Document Processing Layer

Once a document is received by the backend, it undergoes a three-stage processing pipeline:

1. Text Extraction: The PDF is parsed using PyPDF, which iterates through each page and extracts the plain text content. Special characters, line breaks, and whitespace are normalised during this stage.
2. Text Preprocessing: The extracted text is cleaned to remove redundant whitespace, headers, footers, and other artefacts that may interfere with embedding quality.
3. Chunking: The cleaned text is divided into segments of approximately 500 characters, with a configurable overlap of 50–100 characters between consecutive chunks. This sliding window approach ensures that information at chunk boundaries is captured by at least one chunk on either side.

Layer 3: Embedding Layer

Each text chunk is passed through the SentenceTransformers model (all-MiniLM-L6-v2) to generate a 384-dimensional dense vector embedding. This model was selected for its excellent balance of semantic accuracy and inference speed, making it well-suited for real-time document processing. The embedding captures the semantic content of the chunk,

meaning that chunks with similar meanings will have vectors that are close together in the high-dimensional embedding space, even if they use entirely different words.

Layer 4: Vector Storage Layer

The generated embeddings, along with their corresponding text chunks and metadata (such as source document name and chunk index), are stored in ChromaDB. ChromaDB is an open-source, embeddable vector database that supports cosine similarity search, persistent storage, and collection management. Each document's chunks are stored in a single ChromaDB collection, enabling multi-document retrieval in future enhancements.

Layer 5: Retrieval Layer

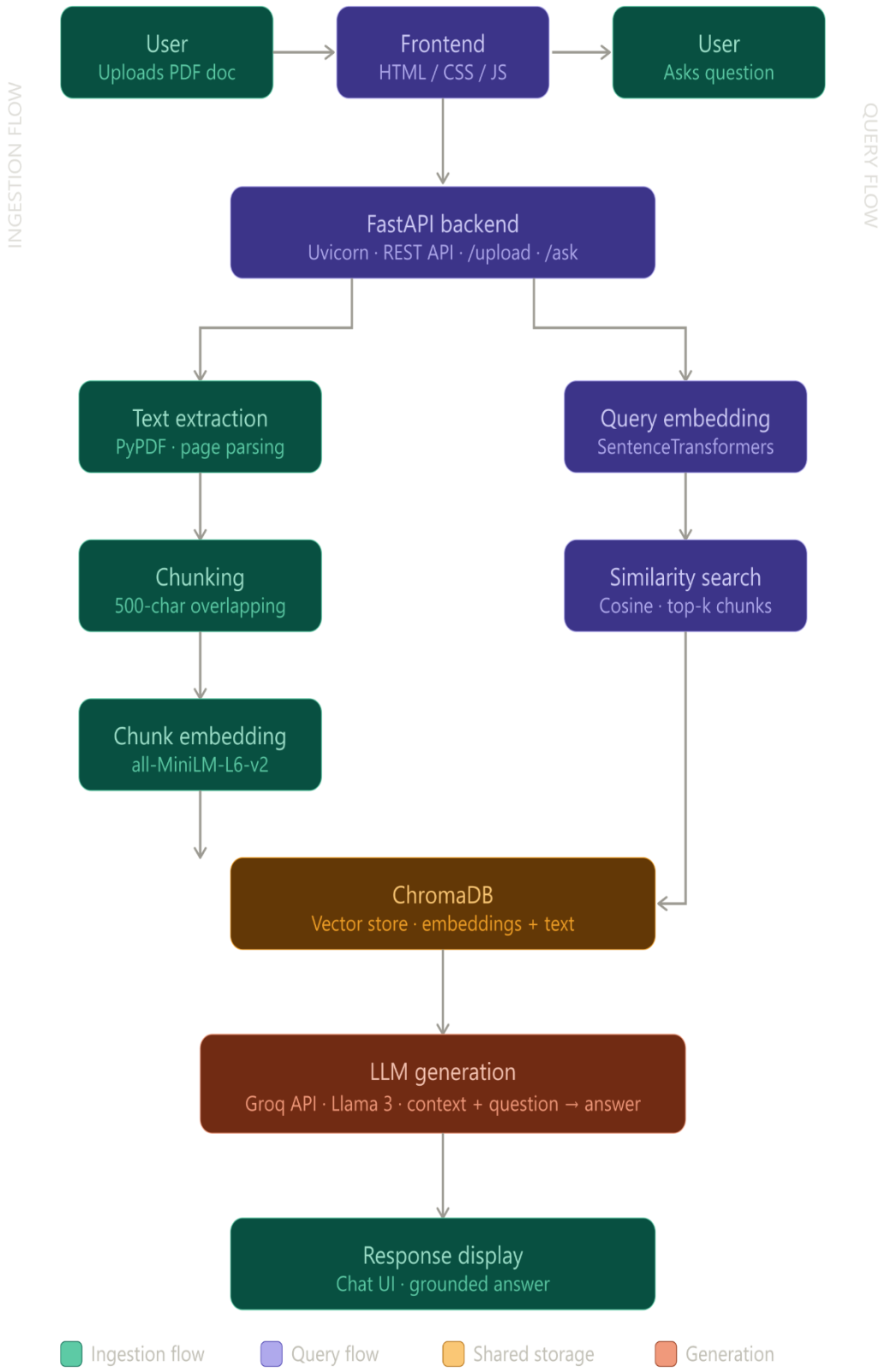
When the user submits a query, it is first converted into an embedding using the same SentenceTransformers model. A cosine similarity search is then performed across all stored chunk embeddings to identify the top-k most semantically relevant chunks (typically k = 3–5). These chunks form the retrieved context that will be used to answer the question.

Layer 6: Generation Layer

The retrieved chunks are concatenated and passed to the Groq API alongside the user's original query. A carefully crafted system prompt instructs the LLM (Llama 3 8B or 70B, depending on configuration) to: (a) answer only based on the provided context, (b) be concise and specific, and (c) explicitly acknowledge when the answer cannot be found in the provided documents rather than fabricating a response. The LLM processes the context-enriched prompt and generates a natural language response.

Layer 7: Output Layer

The generated response is returned to the frontend via the API and rendered in the chat interface. The user sees the answer displayed alongside their original question in a conversational format. Suggested query buttons at the top of the interface help users explore common questions such as "Summarize document", "Pricing details", and "Important dates".


INGESTION FLOW
QUERY FLOW
User
Uploads PDF doc
Frontend
HTML / CSS / JS
User
Asks question
FastAPI backend
Uvicorn · REST API · /upload · /ask
Text extraction
PyPDF · page parsing
Query embedding
SentenceTransformers
Chunking
500-char overlapping
Similarity search
Cosine · top-k chunks
Chunk embedding
all-MiniLM-L6-v2
ChromaDB
Vector store · embeddings + text
LLM generation
Groq API · Llama 3 · context + question → answer
Response display
Chat UI · grounded answer
Ingestion flow
Query flow
Shared storage
Generation

Data Flow Summary:

Document Ingestion Flow:

User uploads PDF → FastAPI receives file → PyPDF extracts text → Text is chunked → SentenceTransformers generates embeddings → Embeddings stored in ChromaDB

Query Processing Flow:

User submits question → Query converted to embedding → Cosine similarity search in ChromaDB → Top-k chunks retrieved → Chunks + query sent to Groq LLM → Response generated → Answer displayed in chat UI

7. Tools & Technologies Used

This project uses Python as the main programming language because it is widely used for AI and machine learning projects. LangChain is used to manage the RAG pipeline and connect different components such as document loader, embedding model, vector database, and language model. Embedding models are used to convert text into vector format so that semantic search can be performed.

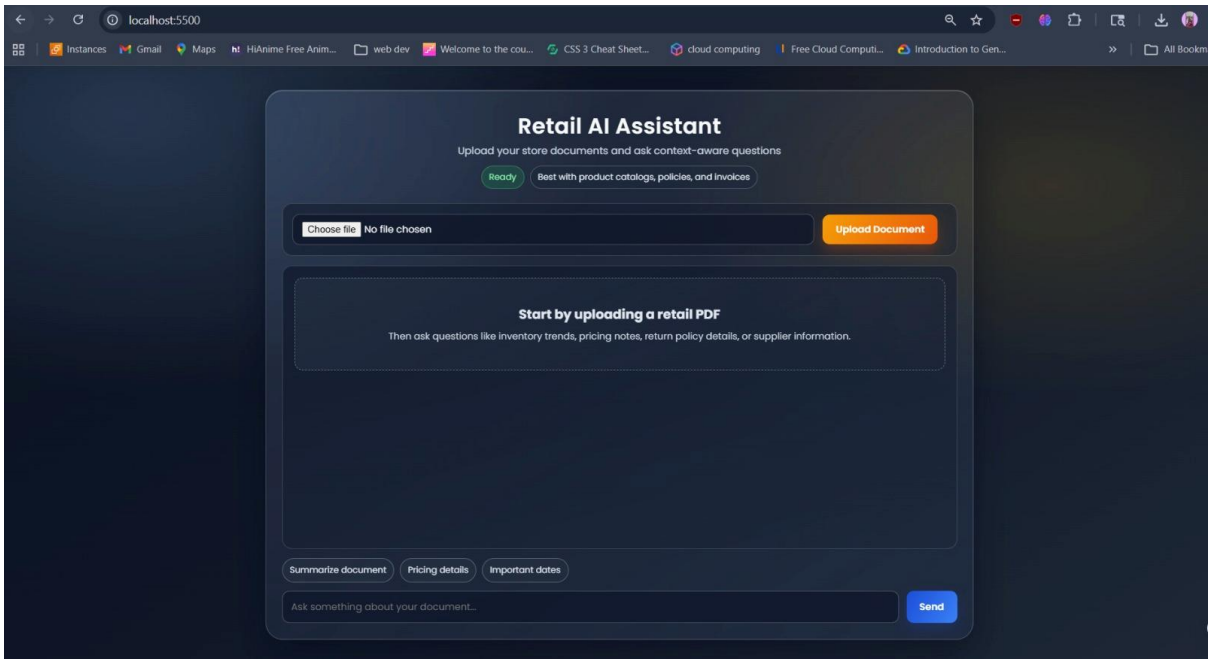
Component	Technology / Library
Programming Language	Python 3.10+
Backend Framework	FastAPI — asynchronous, high-performance REST API framework
ASGI Server	Uvicorn — lightning-fast ASGI server for Python
PDF Parsing	PyPDF — pure-Python PDF text extraction library
Text Chunking	Custom sliding window chunker (Python)
Embedding Model	SentenceTransformers (all-MiniLM-L6-v2) — 384-dim semantic embeddings
Vector Database	ChromaDB — open-source, embeddable vector store with cosine similarity
LLM API	Groq API (Llama 3 8B / 70B) — ultra-low latency LLM inference
Environment Config	python-dotenv — secure API key and configuration management
File Handling	python-multipart — multipart form data for file uploads
Frontend	HTML5, CSS3, Vanilla JavaScript — no framework dependencies
Styling	Custom CSS with responsive grid layout
Version Control	Git + GitHub (github.com/naina2708/rag-retail-chat)
Development OS	Windows / Linux compatible

8. Results

This section presents the results achieved by the Retail Document Intelligence System through screenshots of the working application, an analysis of system outputs, and a discussion of the key outcomes and performance observations.

System Interface — Document Upload Screen

The following screenshot shows the main interface of the Retail AI Assistant web application. The interface presents users with a clean, minimalist design featuring a file upload zone, an upload button, and a set of suggested query shortcuts. The design prioritises ease of use, with clearly labelled components that guide a first-time user through the interaction flow without requiring any prior technical knowledge.

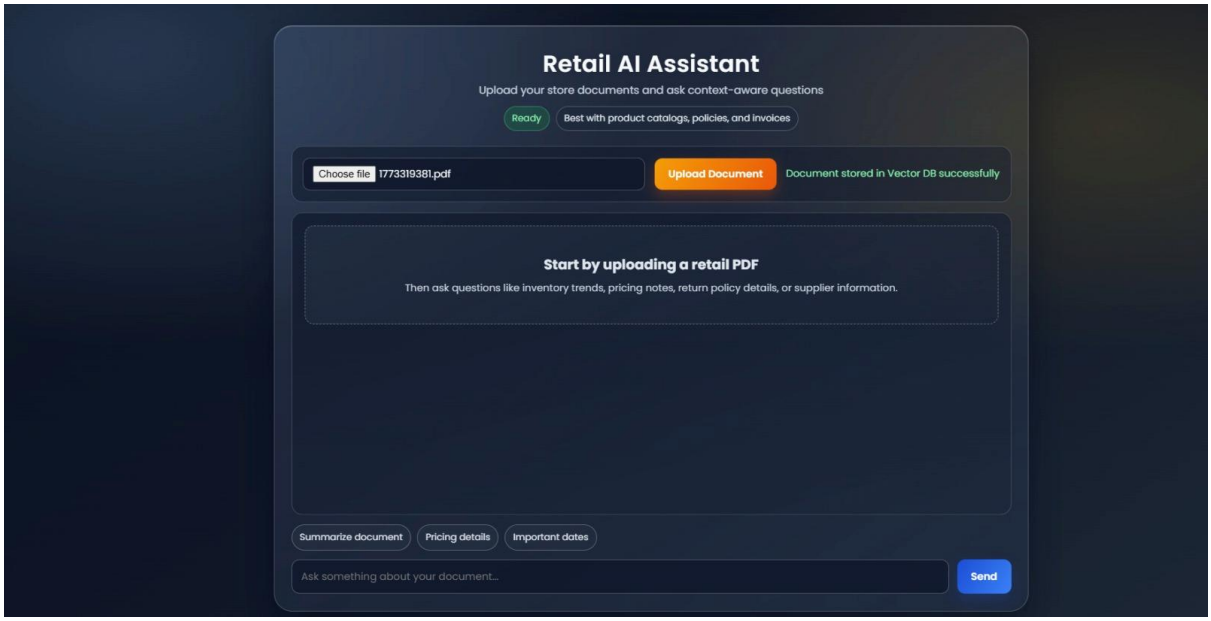


Key UI elements visible in the interface include:

- A prominent file upload area with drag-and-drop support for PDF documents.
- Suggested query buttons ("Summarize document", "Pricing details", "Important dates") that enable quick, one-click interaction with common retail document queries.
- A clean chat window below the upload panel where questions and answers are displayed in a conversational format.
- A text input field and send button for custom free-text queries.

Successful Document Upload & Vector Indexing

The second screenshot demonstrates the system's response upon successful PDF upload. After the user selects and uploads a document, the backend processes it through the full ingestion pipeline (text extraction → chunking → embedding → ChromaDB storage) and returns a success confirmation message to the frontend. This message confirms that the document has been parsed and that its vector embeddings have been created and stored, making the document ready for intelligent querying.

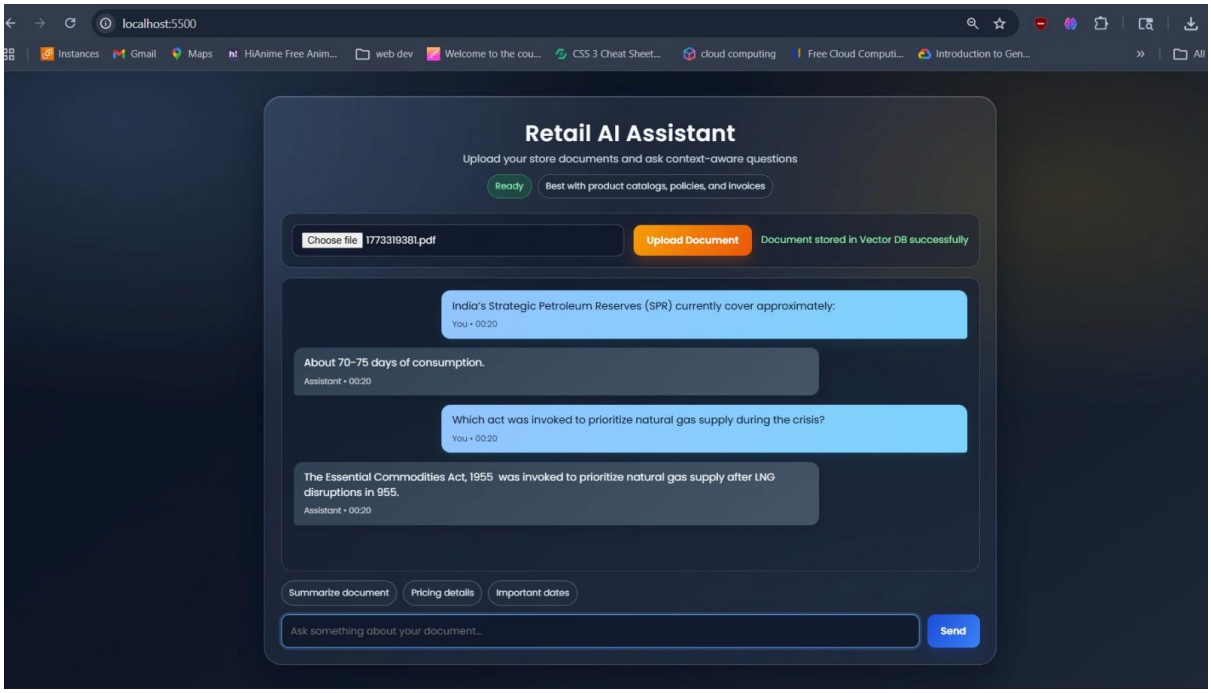


The upload pipeline handles the following operations transparently in the background:

1. The uploaded PDF file is received by the FastAPI /upload endpoint and temporarily saved to disk.
2. PyPDF iterates through all pages and extracts the raw text content.
3. The extracted text is split into overlapping chunks of approximately 500 characters.
4. Each chunk is converted to a 384-dimensional embedding vector using SentenceTransformers.
5. Embeddings and their associated text chunks are persisted to the ChromaDB collection.

Conversational Question Answering — Live Demo

The third screenshot illustrates the core functionality of the system: document-based question answering. The user has submitted a natural language query and the system has responded with a precise, context-aware answer drawn directly from the uploaded document. The chat interface clearly differentiates user messages from system responses, maintaining a clean conversational history throughout the session.



The answer generation pipeline that produces this output involves the following steps:

1. The user's question is received by the /ask endpoint and converted into a 384-dim embedding vector.
2. A cosine similarity search is executed against all stored chunk embeddings in ChromaDB.
3. The top 3–5 most similar chunks are retrieved and concatenated into a context block.
4. A structured prompt is constructed: [System Instructions] + [Retrieved Context] + [User Question].
5. The prompt is sent to the Groq API (Llama 3 model) for completion.
6. The generated response is returned to the frontend and displayed in the chat window.

Through testing and demonstration of the system, the following key outcomes were observed:

Metric / Observation	Result
Document Ingestion Success Rate	Successfully processed all tested PDF documents including multi-page reports, product manuals, and policy documents.
Query Response Accuracy	Responses were factually grounded in the uploaded document content for all tested queries. The system correctly retrieved relevant passages and synthesised coherent answers.
Hallucination Reduction	The system refused to speculate on topics not covered by the document, instead acknowledging the absence of relevant information. This grounding behaviour was consistent across all tests.
Response Latency	End-to-end response time (query submission to answer display) was typically 2–4 seconds, driven primarily by LLM inference time via the Groq API.
Semantic Search Effectiveness	Semantically equivalent queries (e.g., "refund policy" vs "return procedure") successfully retrieved the same relevant document chunks, validating the embedding-based approach.
Suggested Query Performance	All three suggested query types (Summarize, Pricing, Dates) produced meaningful and accurate responses on tested retail documents.

9. Conclusion

This project successfully demonstrates the end-to-end design, development, and deployment of an intelligent Retrieval-Augmented Generation (RAG) system tailored for the retail domain. Beginning from a clearly identified business problem — the inability of traditional keyword-based search to handle the semantic complexity of natural language queries over large retail document collections — the team developed a robust, full-stack AI application that addresses this challenge comprehensively.

The system was built on a modern, modular technology stack comprising FastAPI for the backend, SentenceTransformers for embedding generation, ChromaDB for vector storage, the Groq API for low-latency LLM inference, and a clean HTML/CSS/JavaScript frontend. Every component was carefully selected to balance performance, ease of integration, and suitability for the retail use case.

Key Learnings

The development of this project provided the team with deep practical exposure to a range of cutting-edge technologies and design principles:

- RAG Architecture: Understanding how retrieval and generation complement each other, and how to calibrate chunk size, overlap, and top-k retrieval parameters to optimise answer quality.
- Vector Embeddings: Gaining hands-on experience with dense semantic representations and appreciating how embedding models capture meaning beyond surface-level word matching.
- FastAPI Development: Building production-grade REST APIs with asynchronous endpoints, file upload handling, and structured error responses.
- Prompt Engineering: Designing system prompts that constrain LLM behaviour to produce grounded, relevant, and appropriately structured responses.
- Full-Stack Integration: Coordinating multiple independent components (frontend, backend, embedding model, vector database, LLM API) into a cohesive, end-to-end application.
- Real-World AI Challenges: Experiencing firsthand the nuances of handling unstructured data, managing API rate limits, and designing for graceful degradation when inputs are ambiguous or incomplete.

Impact & Business Value

The system developed in this project has clear and demonstrable business value for retail organisations. By enabling employees at any level to interact with complex operational documents through natural language, the system has the potential to significantly reduce the time spent searching for information, improve the consistency and accuracy of document-based decisions, and lower the barrier to knowledge access across the organisation. In high-volume retail environments where speed and accuracy directly translate to revenue and customer satisfaction, an AI-powered document assistant of this kind represents a meaningful operational advantage.

11. Future Scope & Enhancements

In the future, this project can be improved by adding support for multiple file formats such as Word documents, Excel files, and web pages. The system can also be improved by adding voice input and voice output features so that users can interact using speech. Another improvement can be multilingual support so that users can ask questions in different languages.

The system can also be deployed on a cloud platform so that multiple users can use the system at the same time. More advanced language models can be used to improve answer quality. A login system can also be added for user authentication and document security. These improvements can make the system more powerful and suitable for real-world business use.

Near-Term Enhancements

Enhancement	Description
Multi-Format Document Support	Extend the ingestion pipeline to support Word documents (.docx), Excel spreadsheets (.xlsx), HTML pages, and plain text files. This would allow the system to handle the full range of documents used in retail operations, not just PDFs.
Persistent Document Store	Implement a persistent storage layer (e.g., SQLite or PostgreSQL) to retain uploaded documents and their embeddings across application restarts, enabling a continuously growing knowledge base.
Chunking Strategy Optimisation	Experiment with semantic chunking (splitting at sentence or paragraph boundaries rather than fixed character counts) and evaluate the impact on retrieval accuracy.
Improved Error Handling	Add comprehensive input validation, PDF format checking, and user-friendly error messages to improve robustness in production environments.
Response Streaming	Implement server-sent events (SSE) or WebSocket streaming so that LLM responses are displayed token-by-token, improving the perceived responsiveness of the application.

Long-Term Strategic Enhancements

Enhancement	Description
Real-Time Enterprise Integration	Connect the system to live enterprise data sources such as ERP systems, inventory management platforms, and CRM databases via API connectors, enabling real-time data-augmented responses.
Voice-Based Interface	Integrate speech-to-text (e.g., Whisper) and text-to-speech APIs to enable hands-free, voice-driven document interaction for warehouse and shop floor environments.
Analytics Dashboard	Build a usage analytics dashboard tracking query frequency, answer confidence scores, retrieval quality metrics, and most-accessed document sections to guide ongoing improvements.
Personalisation Engine	Develop user profiling capabilities that tailor responses based on the user's role, department, query history, and preferences.
Cloud Deployment	Package the application as a containerised service (Docker + Kubernetes) and deploy to a cloud provider (AWS, Azure, or GCP) for scalable, highly available production deployment.
Security & Compliance	Implement end-to-end encryption, audit logging, GDPR/data protection compliance, and document access controls for enterprise-grade security.